XYZ-guard — Complete Platform Reference

Core Library + Agent Gateway (Sidecar) + Control Plane

A comprehensive feature and architecture guide, grounded in the XYZ-guard repository (README, PRFAQ, CHANGELOG, launch materials, gateway architecture, and control-plane documentation).

---

1. What XYZ-guard Is

XYZ-guard is a **runtime safety and reliability layer for AI agents**. It intercepts every tool call an AI agent makes, scores its risk on a 0–100 scale, and enforces safety policies — regardless of which framework the agent runs on. It is the guardrail layer between agents and the outside world.

Unlike content-safety tools that inspect what an agent *says* (toxicity, PII, prompt injection at the model input/output level), XYZ-guard operates at the **tool-call / action level** — governing what an agent *does*. It is deterministic: risk scoring and anomaly detection use policy rules, pattern matching, and statistical heuristics, with **no LLM calls during evaluation**, ensuring predictable behavior and zero added inference cost.

The platform has **three layers**, which together form the complete solution:

Layer	What it is	Deployment
1. Core Library (`agentlasso`)	In-process Python guardrail — the Guard engine, policy engine, anomaly detectors, circuit breaker, checkpoint/rollback, adapters	`pip install agentlasso`
2. Agent Gateway (the "Sidecar")	Language-agnostic HTTP proxy that externalizes the core library so any-language agents get guardrails with zero safety code	One Docker image, three deployment modes
3. Control Plane	Centralized web console + API that manages a fleet of gateways — pushes config, monitors, tracks cost, runs	React UI + FastAPI backend

experiments

- **License:** Apache 2.0 · **Distribution:** PyPI + GitHub
- **Maturity:** Beta (v0.1.0, 2026-06-22); 585 tests (unit, property-based, integration), 90%+ coverage, strict mypy, CI across Python 3.10–3.13 on Ubuntu/macOS/Windows.

---

2. Layer 1 — The Core Library

The core library is the in-process foundation. It can be used entirely on its own (5-line integration) or wrapped by the Agent Gateway.

2.1 Guard — the central mediator

`Guard.validate(action, params)` is the single entry point. Each call runs a deterministic pipeline:

1. **Adapter pre-hooks** — normalize action/params from the framework-specific format
2. **Policy engine** — evaluate rules; decide allow / block / score
3. **Anomaly detection** — check for loops, drift, hallucination, contradictions
4. **Gateway (scoring)** — combine signals into a final risk score and tier
5. **Intervention** — if tier = intervene, request human approval (callback/webhook/dashboard)
6. **Trace storage** — record the full evaluation result
7. **Circuit breaker** — update failure counters

The result carries a `tier` of `allow`, `intervene`, or `block`. Blocked actions raise `ActionBlockedError`. Policy evaluation typically adds **under 5 ms** per tool call.

2.2 Policy engine

YAML-based rules that **allow, score, or block** actions by pattern:

- **`allow`** patterns — always permitted (score 0)
- **`block`** patterns — always blocked, glob syntax e.g. `*.delete`, `*.drop` (score 100)
- **`risk_adjust` rules** — add/subtract points for matching actions (e.g. `email.send` +25, `file.write` +15, `db.query` +5)
- **Thresholds** — `allow_max` (default 30) and `intervene_max` (default 70) map scores to tiers
- **Hot-reload** — policies reload from **file, HTTPS, or S3** with no restart

2.3 Risk scoring & three tiers

Every action gets a **0–100 risk score** mapped to three tiers — the platform's signature capability:

Score	Tier	Behavior
≤ 30	**allow**	Proceeds automatically
31–70	**intervene**	Routed to a human for approval (the "middle ground" other tools lack)
> 70	**block**	Rejected; raises `ActionBlockedError`

2.4 Anomaly detection

Four built-in deterministic detectors catch problematic agent behavior:

- **Loop detection** — agent repeating the same action
- **Drift detection** — agent deviating from expected behavior patterns
- **Hallucination detection** — agent referencing non-existent resources
- **Contradiction detection** — agent actions that conflict with prior context

2.5 Reliability primitives

- **Circuit breaker** — automatically trips when error/block rates exceed a configurable failure threshold; supports recovery timeout and half-open state to prevent cascading failures from a runaway agent
- **Checkpoint / rollback** — save agent state at safe points and restore when things go wrong
- **Intervention gateway** — human-in-the-loop approval for medium-risk actions via callbacks, webhooks, or the dashboard
- **Health checker** — dependency status reporting

2.6 Multi-framework adapters

Pluggable, drop-in adapters mean policies stay identical across LLMs/frameworks. Shipped adapters: **OpenAI, Anthropic Claude, AWS Bedrock, Strands Agents**. A custom adapter for any framework takes under ~50 lines. Multiple adapters can run at once. Installed as extras: `agentlasso[openai|claude|bedrock|strands|dashboard|tui|all]`.

2.7 Developer ergonomics

- **`@protect` decorator** — guard any function inline
- **CLI** — `validate`, `traces`, `report`, `tui`, `dashboard` commands
- **Terminal UI** (Textual) — rich local-development monitoring
- **Web dashboard** (FastAPI) — real-time trace viewer, policy editor, agent metrics, intervention queue (`agentlasso dashboard --port 8420`)

2.8 Observability & storage

All evaluations are traced and stored — action, params, risk score, decision tier, anomaly flags, timestamps. Backends: **SQLite** (local) or **PostgreSQL** (production), with a **redaction**

filter and **real-time WebSocket streaming**. Runs entirely in-process — no telemetry, no phone-home, no third-party services.

---

3. Layer 2 — The Agent Gateway (Sidecar)

The Agent Gateway externalizes the core library. The agent **never calls tools directly** — it calls the gateway, which validates and then proxies to the real tool. The agent developer writes **zero safety code** and can be written in **any language** (Java, Go, C, JS, Python).

Naming: The UI now calls this an **"Agent Gateway"** rather than "Sidecar," because it supports three deployment modes, not just per-pod K8s. API routes still use `/api/sidecars` for backward compatibility.

3.1 Three deployment modes (one Docker image)

Mode	Description	Best for
Sidecar (per-pod)	One gateway per pod, co-located with the agent in K8s	Strong isolation, per-agent network policy
Shared gateway	One gateway serving multiple agents with per-agent auth	Cost efficiency, small teams, dev environments
Global NAT gateway	Network-level proxy all agents route through	Enterprise-wide governance, zero agent config

A dedicated **transparent proxy mode** (DNS / service mesh such as Istio or Envoy) requires zero application changes — "the agent doesn't even know the sidecar exists."

3.2 Two request paths — the key upgrade

The gateway is more than a proxy; it is an **Intent-Invoke Orchestrator** with a single entry/exit point and two paths:

- **PATH 1 — Direct tool call (`POST /tool/{action}`)**: the agent already knows the tool. Flow: Orchestrator → Agent Auth → Quota → Policy → Tool Router → execute → trace + cost. No LLM involved inside the gateway.
- **PATH 2 — LLM-driven agentic loop (`POST /invoke`)**: the agent sends *intent*. Flow: Orchestrator → Auth (Gate 1: may the agent use `/invoke`?) → Quota #1 (pre-LLM cost estimate) → **ABC-LLM generates a tool plan** → Auth (Gate 2: per-tool grants) → Policy (per-tool) → Tool Router executes each → **ABC-LLM synthesizes the final response** → deliver + report costs.

3.3 The ABC-LLM engine (LLM gateway inside the gateway)

- **Multi-provider routing** with smart routing, **fallback chains**, and **A/B experiments**
- **Frontier model access** — Claude, GPT, Bedrock (6 providers)
- Powers PATH 2's plan-generation and response-synthesis rounds

3.4 Tool Router — three resolution targets

After policy approves an action, the Tool Router resolves *where* to send it:

- **HTTP services** (email, DB, CI/CD)
- **MCP servers** (GitHub, filesystem — embedded, remote HTTP, or stdio subprocess, with auto-discovery)
- **Agent-as-Tool** (e.g. research, writer sub-agents)

3.5 Dual quota enforcement (runtime, sidecar-local)

Quotas are **actively enforced on the sidecar**, not just informational:

- **Quota #1 (pre-LLM)** — can the agent afford the intent before any LLM call?
- **Quota #2 (per-tool)** — can the agent afford each tool the plan invokes?
- Enforces **rate limits, budget caps, model allowlists, and max_tokens caps**
- **Pre-request budget projection** blocks *before* calling the LLM using local per-model pricing tables (no round-trip)
- 429 for rate/budget, 403 for model restrictions, silent cap for max_tokens
- Budget alerts at 80% / 90% / 100%

3.6 Per-agent tool authorization (least privilege)

When multiple agents share a gateway: each agent gets explicit tool grants (exact names or wildcards like `github.*`); unregistered agents are denied by default; `*` grants full access. (JWT-based temporary elevation is documented as a future capability.)

3.7 Intent caching

Exact-match intent cache to reduce repeat-evaluation cost, with `/cache/stats` for hit/miss. (Template/auto caching is planned.)

3.8 Gateway configuration API

Endpoint	Method	Purpose
`/config`	POST/GET	Apply / view full config (tools + policy + quota + auth + MCP)
`/config/tools`, `/config/tools/{name}`	POST/DELETE	Replace, add, update, or remove tools
`/config/policy`	POST	Replace the policy
`/config/quota`, `/config/quota/reset-`	POST	Apply quota; reset spend

`spend`		counter
`/config/agent-auth`	POST/GET	Configure per-agent tool grants
`/config/mcp-servers`, `/{name}`, `/{name}/refresh`	POST/DELETE	Add, remove, re-discover MCP servers
`/config/llm`	POST	Update LLM routing, models, credentials
`/tool/{action}` · `/invoke`	POST	Direct tool call (PATH 1) · agentic loop (PATH 2)
`/tools` · `/models` · `/cache/stats` · `/health`	GET	Introspection and health

---

4. Layer 3 — The Control Plane

The control plane is a centralized web console (React + Tailwind UI on port 3000, FastAPI backend on port 8400, SQLite/Postgres) that manages a **fleet** of Agent Gateways. It pushes configuration over HTTP and receives traces and usage back. Without it, each gateway would require manual SSH, file edits, and restarts; with it, a policy change is pushed to every gateway in **under a second, no restart**.

4.1 Twelve core capabilities

#	Capability	What it does
1	**Agent Gateway Registry**	Register gateways, monitor health, push config to one or all; supports sidecar / shared / NAT modes
2	**Agent Registry**	View all agents across the fleet — framework badges, tools, model, gateway assignment (demo: 9 agents across 7 frameworks — OpenAI, Anthropic, Strands, Bedrock, AgentCore, CrewAI, LangGraph)

3	**Tool Management**	CRUD tools per gateway, with search/filter by name, description, endpoint, or gateway
4	**Policy Management**	Create / edit / push YAML allow-block-risk policies
5	**MCP Server Management**	Configure MCP servers (embedded, remote HTTP, stdio) with auto-discovery
6	**Model Configuration**	Central LLM model registry (14 pre-seeded from ABC-LLM) with providers, pricing, capabilities (tools/vision), category (reasoning/general/speed), and inline routing-strategy editing
7	**Quotas (runtime enforced)**	Rate limits, budget caps, model allowlists, max_tokens — pushed to gateways for active enforcement, not just dashboards
8	**Sandbox**	Three-tab testing: Chat (invoke LLM via gateway), Scenarios (one-click demos), Code (write/run agent code) — via the gateway proxy to eliminate CORS
9	**Cost Dashboard**	LLM spend by model, gateway, agent, and day
10	**A/B Experiments**	Percentage-based traffic splitting between models with side-by-side results
11	**Live Trace Viewer**	Real-time WebSocket feed of tool calls + `/invoke` calls across all gateways,

		with session/plan/step grouping and parameters
12	**Audit Log**	Immutable record of who changed what config, when (filterable by resource, action, actor)

4.2 Runtime enforcement, not just reporting

A key architectural distinction: quotas and budgets are **runtime-enforced on the gateway**. Creating a quota only saves it in the database; calling `/push` activates enforcement. The gateway then loads the quota into its in-memory enforcer, checks every request, and rejects/caps those exceeding limits — calculating cost locally (no round-trip) with pre-request budget projection to prevent overshoot.

4.3 Control-plane API surface

The backend exposes REST endpoints for **Sidecars/Gateways** (register, push, push-all, health), **Tools**, **Policies** (incl. `/push`), **MCP Servers**, **Models**, **Quotas** (incl. `/push` to activate), **Cost Tracking** (summary, records, single/batch record), **A/B Experiments** (create, toggle, results), **Live Traces** (ingest, recent, `/ws/traces` WebSocket), **Audit Log**, **Agents**, a **Sidecar Proxy** (`/api/proxy/sidecar/{id}/{path}` — forwards UI requests, eliminating CORS), and **Health**.

4.4 UI organization

Warm light theme (stone-50, white cards, Inter, violet primary). Sidebar sections: **Overview** (Dashboard), **Infrastructure** (Gateways, Agents, Tools, MCP Servers), **Safety** (Policies, Quotas, Audit Log), **Intelligence** (Models, Costs, A/B Tests, Efficiency), **Monitor** (Live Traces), **Develop** (Sandbox) — following the operator's mental model: what's happening → what exists → what's enforced → LLM intelligence & cost → observability → test & build.

4.5 Persistence & demo fleet

State persists in SQLite (`control_plane.db`) plus JSON state, restored on startup (no re-seeding). The demo fleet showcases 7 gateways, 9 agents (7 frameworks), 22 tools, 5 policies, 5 quotas, and 5 tracked models with live traces.

---

5. End-to-End Architecture

Figure 1: The three layers end-to-end — agents call the gateway; the gateway enforces auth, quota, and policy, then routes to tools/LLMs; the control plane pushes config and collects traces and cost.

Bypass-proof by design: network policy ensures agents can reach only their gateway, never the backend services directly — so policy enforcement cannot be skipped.

5.1 The Two Request Paths

<image: request_paths.png>

Figure 2: PATH 1 is a direct tool call (no LLM in the gateway). PATH 2 is the LLM-driven agentic loop — plan generation, dual authorization (Gate 1 for /invoke, Gate 2 per tool) and dual quota, tool execution, then response synthesis.

6. What Changed vs. the First (Sidecar-Only) Document

This revision captures the **full platform**, adding the material the earlier sidecar-only summary missed:

- **The core library layer** in full — Guard pipeline, four anomaly detectors, circuit breaker, checkpoint/rollback, `@protect`, CLI, TUI, adapters, redaction, SQLite/Postgres storage.
- **The two-path gateway model** — the `/invoke` LLM-driven agentic loop, the Intent-Invoke Orchestrator, and dual (Gate 1 / Gate 2) authorization.
- **The ABC-LLM engine** — smart routing, fallback, A/B, frontier-model access.
- **Tool Router with Agent-as-Tool** resolution alongside HTTP and MCP.
- **Dual quota enforcement** with pre-LLM budget projection and local pricing.
- **The complete control plane** — all 12 capabilities, the full REST API surface, runtime-enforcement architecture, UI structure, persistence, and demo fleet.
- **Product framing** — positioning vs. Bedrock Guardrails (content vs. action), deterministic/no-LLM evaluation, in-process/no-phone-home, and Beta production-readiness metrics.

Sources: XYZ-guard repository — `README.md`, `PRFAQ.md`, `CHANGELOG.md`, `launch-materials.md`, `docs/gateway-architecture.md`, `sidecar/docs/architecture.md`, `control-plane/README.md`, and `control-plane/docs/getting-started.md`.